

Map Based EKF Localisation - Part 1

Vasutorn Siriyakorn, and Mohammad Haadi Akhter

October 26, 2025

Abstract

This lab was aimed at developing an understanding of how to implement a variation of the Bayes Filter, namely the Extended Kalman Filter (EKF), which serves as an approximate nonlinear extension of the Kalman Filter (KF). The objective was to apply this algorithm to localize a differential drive robot within a simulation environment. The robot model designed in the previous lab was utilized for this purpose.

In this lab, the equations and variables associated with the EKF were derived and implemented using a Python-based program. The results of the EKF-based localization were plotted, demonstrating that the method effectively estimates the robot's position and orientation. Additionally, the effect of different measurement update rates on localization accuracy was analyzed, showing that higher update rates yield results closer to the ground truth. The issue of angle wrapping in orientation calculations was also addressed to ensure continuous and accurate estimation of the robot's heading.

1 Introduction

The Pre-Laboratory session introduced the foundational concepts of random variables and Gaussian distributions, emphasizing their central role in modeling numerous natural and engineered processes. A Gaussian distribution is fully defined by its mean and variance in the scalar case, and by a mean vector and covariance matrix in the multivariate case, the latter capturing dependencies among variables. The session also covered the derivation of mean and covariance for linear transformations and products of Gaussian random variables, establishing the mathematical groundwork for probabilistic modeling.

Building on these concepts, the session explored their application in robotics, where system states and motion dynamics are represented probabilistically within a state-space framework. Under the assumptions that both state transition and observation models are linear with additive Gaussian noise, the Kalman Filter (KF) algorithm—formulated by Rudolf Emil Kalman—enables optimal state estimation. The algorithm iteratively predicts the next state based on prior knowledge and control inputs, then refines this prediction through observations, yielding updated estimates of the state's mean and covariance.

However, real robotic systems are predominantly non-linear, and therefore, the assumption of linearity in the motion and observation models is not practical. To address this limitation, the Extended Kalman Filter (EKF) was introduced. This algorithm extends the applicability of the Kalman Filter to non-linear systems by linearizing the non-linear motion

and observation models around the mean of the previous state using a Taylor series expansion and Jacobian matrices. Once these models are linearized, the mean and covariance can be computed using the properties of Gaussian distributions and the equations of the EKF.

Finally, the application of the Kalman Filter and the Extended Kalman Filter in robot localization was introduced. In this context, the filters predict the robot’s next pose based on the previous pose and the motion model, then refine this prediction using observations from the environment, thereby improving the accuracy of state estimation.

This reports is organized as follows: Section 2 describes how to simulate the robot by implementing the *DifferentialDriveSimulatedRobot* class inheriting from the *SimulatedRobot* base class. This section also explains how to use the class to simulate the trajectories requested in the lab. Section 3 introduces the theoretical foundations of the Kalman Filter and the Extended Kalman Filter (EKF) algorithms. Section 4.1 details the specific implementation of the EKF for localizing the 3-DOF differential drive robot, including the specific motion and measurement models used. The section 5 is devoted to the description of the problems found during the lab. Finally, ?? summarizes the results and findings of the EKF localization.

2 Simulation

The simulation development begins by implementing a base class for the robot, including its state. The differential drive mobile robot has six key states which contain the pose and velocity of the robot as follow

$$\begin{aligned}\mathbf{x}_{s_k} &= \begin{bmatrix} {}^N x_{s_k} & {}^N y_{s_k} & {}^N \psi_{s_k} & {}^B u_{s_k} & {}^B v_{s_k} & {}^B r_{s_k} \end{bmatrix}^\top \\ &= \begin{bmatrix} {}^N \boldsymbol{\eta}_{s_k} & {}^B \boldsymbol{\nu}_{s_k} \end{bmatrix}^\top\end{aligned}\tag{1}$$

The pose vector at time step k , denoted as ${}^N \boldsymbol{\eta}_{s_k}$, comprises the robot’s position along the x-axis (${}^N x_{s_k}$), y-axis (${}^N y_{s_k}$), and its heading, represented by the rotation about the z-axis (${}^N \psi_{s_k}$); all components are expressed in the global reference frame $\{N\}$. Correspondingly, the velocity vector at time step k , ${}^B \boldsymbol{\nu}_{s_k}$, contains the robot’s linear velocity along the x-axis (${}^B u_{s_k}$), linear velocity along the y-axis (${}^B v_{s_k}$), and angular velocity about the z-axis (${}^B r_{s_k}$), with each defined in the robot reference frame $\{B\}$.

Since the pose and velocity vectors are represented in difference coordinate frames, compounding operations—which are denoted by the Oplus ($\oplus$) and Ominus ($\ominus$) operators—are employed to facilitate vector addition and subtraction across frames. These operations, implemented using the **NumPy** library, are essential for forward kinematics in robotic simulation, providing a systematic means of combining vectors formulated in different reference frames. The Oplus ($\oplus$) operation can be formally expressed as follows:

$$\begin{aligned}
{}^A\mathbf{x}_C &= {}^A\mathbf{x}_B \oplus {}^B\mathbf{x}_C \\
&= \begin{bmatrix} {}^Ax_B \\ {}^Ay_B \\ {}^A\psi_B \end{bmatrix} \oplus \begin{bmatrix} {}^Bx_C \\ {}^By_C \\ {}^B\psi_C \end{bmatrix} \\
&= \begin{bmatrix} {}^Ax_B + {}^Bx_C \cos({}^A\psi_B) - {}^By_C \sin({}^A\psi_B) \\ {}^Ay_B + {}^Bx_C \sin({}^A\psi_B) + {}^By_C \cos({}^A\psi_B) \\ {}^A\psi_B + {}^B\psi_C \end{bmatrix}
\end{aligned} \tag{2}$$

Where ${}^A\mathbf{x}_C$ is the state vector from $\{A\}$ to C express in frame $\{A\}$, ${}^A\mathbf{x}_B$ is the state vector from $\{A\}$ to B express in frame $\{A\}$, and ${}^B\mathbf{x}_C$ is the robot pose vector from $\{B\}$ to C express in frame $\{B\}$. In the other hand, Ominus ($\ominus$) operation as follow

$${}^B\mathbf{x}_A = \ominus {}^A\mathbf{x}_B = \begin{bmatrix} -{}^Ax_B \cos({}^A\psi_B) - {}^Ay_B \sin({}^A\psi_B) \\ {}^Ax_B \sin({}^A\psi_B) - {}^Ay_B \cos({}^A\psi_B) \\ -{}^Az_B \\ -{}^A\psi_B \end{bmatrix} \tag{3}$$

Where ${}^B\mathbf{x}_A$ is the pose vector from $\{B\}$ to $\{A\}$ express in $\{B\}$ and ${}^A\mathbf{x}_B$ is the pose vector from $\{A\}$ to $\{B\}$ express in $\{A\}$.

2.1 Differential Drive Robot Simulation

Moving on to the differential drive simulation robot implementation, we derive a class called **DifferentialDriveSimulatedRobot** from **SimulatedRobot** class whose template was provided to us. We override the **fs** method of the super class simulation using the forward kinematics equations with constant velocity assumption as follow,

$$\mathbf{x}_{s_k} = \begin{bmatrix} {}^N\boldsymbol{\eta}_{s_k} \\ {}^B\boldsymbol{\nu}_{s_k} \end{bmatrix} = f(\mathbf{x}_{s_{k-1}}, \mathbf{u}_{s_k}, \mathbf{w}_{s_k}) \tag{4}$$

$$= \begin{bmatrix} {}^N\boldsymbol{\eta}_{s_{k-1}} \oplus ({}^B\boldsymbol{\nu}_{s_{k-1}} \Delta t + \frac{1}{2} \mathbf{w}_{s_k} \Delta t^2) \\ {}^B\boldsymbol{\nu}_{s_{k-1}} + \mathbf{K}({}^B\boldsymbol{\nu}_d - {}^B\boldsymbol{\nu}_{s_{k-1}}) + \mathbf{w}_{s_k} \Delta t \end{bmatrix} \tag{5}$$

$$\mathbf{u}_k = [u_d \quad r_d]^\top, \tag{6}$$

$${}^B\boldsymbol{\nu}_d = [u_d \quad 0 \quad r_d]^\top \tag{7}$$

$$\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}_w), \tag{8}$$

$$\mathbf{Q} = \begin{bmatrix} \sigma_u^2 & 0 & 0 \\ 0 & \sigma_v^2 & 0 \\ 0 & 0 & \sigma_r^2 \end{bmatrix} \tag{9}$$

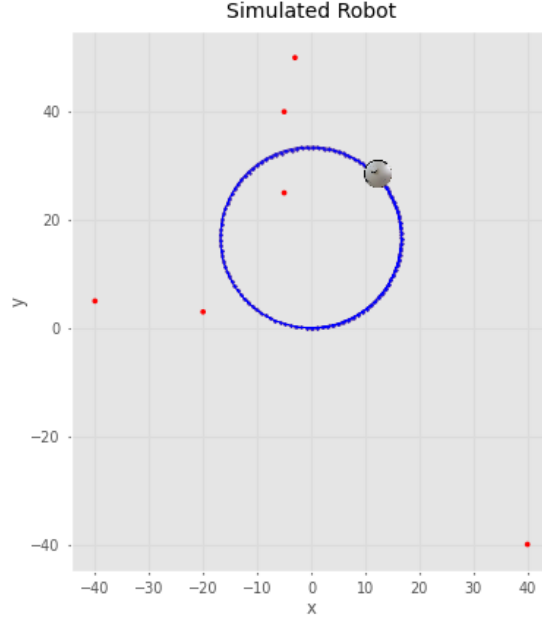


Figure 1: Simulation of the Differential Drive Robot

Where, $K = \text{diag}(k_1, k_2, k_3)$, $k_i > 0$ is a diagonal matrix containing the gains used to drive the simulated velocity towards the desired input velocity. $\mathbf{u}_k$ is input vector of the robot consist of the desired linear velocity along x-axis (u_d), linear velocity along y-axis (v_d) and angular velocity about z-axis (r_d). ${}^B\boldsymbol{\nu}_d$ is the desired velocity vector of the robot. Last, $\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}_w)$ is the zero mean gaussian noise vector for the forward kinematics with constant velocity assumption, or can be implied as a acceleration. The implement those equations in code where also utilize the previously define **Pose3D** class and it's methods in estimating the next pose for the simulation given a velocity control command. We then use the **PlotTrajectory** function defined inside the **Localization** class to verify the simulation of the differential drive robot. In the resulting plot, the robot's state is set to the simulated state, enabling visualization of the simulated robot's trajectory as shown in Figure 1.

As shown in the Figure 1 trajectory of the simulated robot in an environment with acceleration noise are shown. The blue curve shows the robot's noisy path, while red dots indicate static landmarks in the workspace. Acceleration noise results in small deviations from the ideal circular trajectory, illustrating the impact of motion uncertainty on robot navigation.

Furthermore, to ensure a comprehensive simulation of the robotic system, the wheel encoder sensor—which provides velocity measurements—was also simulated. This enables realistic modeling of the robot's perception of motion based on encoder feedback, thereby improving the fidelity of the overall simulation.

2.2 Encoder Sensor Simulation

The simulation of encoders implemented in **ReadEncoders** method of the **DifferentialDriveSimulatedRobot**, where the sensors provide only the number of pulse of the encoder with some sensors noise. To implement the encoder, the simulated linear and angular velocity are mapped to the encoder pulse by the following equation.

$$v_{r,s_k} = B u_{s_k} + \left(\frac{B r_{s_k} \cdot L}{2} \right) \quad (10)$$

$$v_{l,s_k} = B u_{s_k} - \left(\frac{B r_{s_k} \cdot L}{2} \right) \quad (11)$$

The (14) using to convert the velocity of the robot to the wheel velocity. Where, v_{r,s_k} and v_{l,s_k} are linear velocity of the right and left wheel, respectively, and $L = 0.5$ is the distance between right and left wheel. Then, pulse of the encoders are calculated by the velocity of the wheels by using follow equation

$$\mathbf{z}_{en,s_k} = \begin{bmatrix} n_{r,k} \\ n_{l,k} \end{bmatrix} \quad (12)$$

$$n_{r,k} = \frac{v_{r,s_k}}{2\pi \cdot R_w \cdot f_{encoder}} \cdot n_{2\pi} + v_{r,s_k} \quad (13)$$

$$n_{l,k} = \frac{v_{l,s_k}}{2\pi \cdot R_w \cdot f_{encoder}} \cdot n_{2\pi} + v_{l,s_k} \quad (14)$$

Where, $f_{encoder} = 1$ is the sampling interval, $R_w = 0.1$ is the wheel radius, $n_{2\pi} = 4096$ is number of pulse for each turn of the wheel, and $n_{r,k}$ and $n_{l,k}$ are number of pulse for each encoder with sensors noise. The noise for this sensor can be modeled as a vector of the random variable $\mathbf{v}_{s_k} \sim \mathcal{N}(0, \mathbf{R}_{s_k})$ which $\mathbf{v}_{en,s_k} = [v_{r,s_k}, v_{l,s_k}]^\top$ and $\mathbf{R}_{en,s_k} = \text{diag}(\sigma_{n_L}^2, \sigma_{n_R}^2)$.

2.3 Compass Simulation

The simulation of the compass is implemented in **ReadCompass** method of **DifferentialDriveSimulatedRobot**. The sensor provides the current yaw of the robot as the reading with some added gaussian noise. The implementation is directly governed by the following equation:

$$\mathbf{z}_{com,s_k} = \psi_{s_k} + \mathbf{v}_{com,s_k} \quad (15)$$

Where, $\mathbf{z}_{com,s_k}$ is the output of the compass that consist of the heading of simulated robot with noise. The noise in the compass reading is modeled using a random variable variable $\mathbf{v}_{com,s_k} \sim \mathcal{N}(0, \mathbf{R}_{com,s_k})$ and $\mathbf{R}_{com,s_k} = \sigma_{com}^2$. The frequency of this sensor is varied to see the effects of the absence of update step in the localization implemented using EKF.

2.4 System diagram of Differential Drive Robot Simulation

In summary, differential drive robot simulation system can be illustrate as in the Figure 2

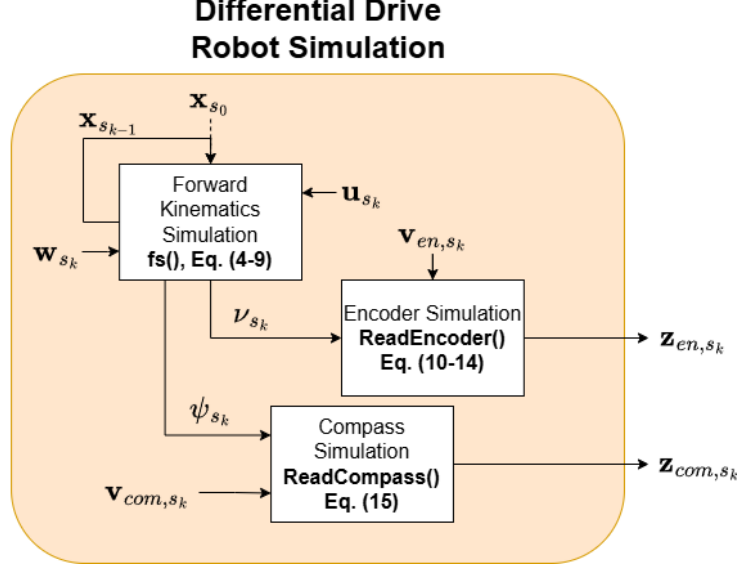


Figure 2: System diagram of Differential Drive Robot Simulation

3 EKF Localization

We first start with the formulations and equations involved in the Kalman Filter(KF). Assuming that the state and the measurement are random variables following a Gaussian distribution, we can define our discrete linear system using the following generalized equations:

$$\bar{\mathbf{x}}_k = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k \quad (16)$$

$$\mathbf{z}_k = \mathbf{H}_k \bar{\mathbf{x}}_k + \mathbf{v}_k \quad (17)$$

where:

- n : Number of states
- l : Number of inputs
- m : Number of measurements
- $\mathbf{A}_k$: Matrix $(n \times n)$ that describes how the state evolves from time $k - 1$ to k without controls or noise.
- $\mathbf{B}_k$: Matrix $(n \times l)$ that describes how the control $\mathbf{u}_k$ changes the state from $k - 1$ to k .
- $\mathbf{H}_k$: Matrix $(m \times n)$ Describes the output $\mathbf{z}_k$ that would be observed from the current state $\mathbf{x}_k$ in absence of noise.
- $\mathbf{x}_{k-1}$: State vector $(n \times 1)$. State of the system at time step $k - 1$.
- $\bar{\mathbf{x}}_k$: State vector $(n \times 1)$ predicted by the LGS at time step k .

- $\mathbf{u}_k$: Input vector ($l \times 1$).
- $\mathbf{z}_k$: Measurement vector of the system ($m \times 1$).
- $\mathbf{w}_k$: Uncertainty of the dynamical model ($n \times 1$).
- $\mathbf{v}_k$: Uncertainty of the observation (output) model ($m \times 1$).

Hence, the KF gives us the equations for computing the mean and covariance of the next state through the following algorithm:

Algorithm 1 Kalman Filter (KF) Algorithm

$\hat{\mathbf{x}}_k \leftarrow \hat{\mathbf{x}}_0, \bar{\mathbf{P}}_k \leftarrow \bar{\mathbf{P}}_0$ ▷ Initial State mean and covariance

while true **do**

Predict Next State Mean

$$\hat{\mathbf{x}}_k = \mathbf{A}_k \hat{\mathbf{x}}_{k-1} + \mathbf{B}_k \mathbf{u}_k \quad (18)$$

Predict Next State Covariance

$$\bar{\mathbf{P}}_k = \mathbf{A}_k \mathbf{P}_{k-1} \mathbf{A}_k^\top + \mathbf{Q}_k \quad (19)$$

if Measurement ($\mathbf{z}_k$) available **then**

Calculate Kalman Gain (K)

$$\mathbf{K}_k = \bar{\mathbf{P}}_k \mathbf{H}_k^\top (\mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{R}_k)^{-1} \quad (20)$$

Update State Mean

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H}_k \hat{\mathbf{x}}_k) \quad (21)$$

Update State Covariance

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \bar{\mathbf{P}}_k \quad (22)$$

end if
end while

where:

- $\hat{\mathbf{x}}_{k-1}$: A posteriori state estimate at time step $k - 1$ ($n \times 1$)
- $\hat{\mathbf{x}}_k$: A priori (predicted) state estimate at time step k ($n \times 1$)
- $\hat{\mathbf{x}}_k$: A posteriori state estimate at time step k ($n \times 1$)
- $\mathbf{P}_{k-1}$: A posteriori error covariance matrix at time step $k - 1$ ($n \times n$)
- $\bar{\mathbf{P}}_k$: A priori (predicted) error covariance matrix at time step k ($n \times n$)

- P_k : A posteriori error covariance matrix at time step k ($n \times n$)
- Q_k : Process noise covariance matrix at time step k ($n \times n$)
- R_k : Measurement noise covariance matrix at time step k ($m \times m$)
- K_k : Kalman gain matrix at time step k ($n \times m$)
- I : Identity matrix of appropriate dimension ($n \times n$)

The equations from (18) and (19) can be considered as a predict step, in the sense of predicting the next state of the input, just on the basis of the given input and the previous state. The (21 - 22) can be regarded as an update step, in the sense of updating and refining the prediction using the reading from the sensor.

However, in some cases the system follows is a non-linear motion model, which is given in section 4.2, therefore, we will have to adhere to the EKF for solving our localization problem to keep the properties of the Gaussian when propagate random variables though non-linear equation. The key concept of the EKF is use the local-linearization algorithm to propagate the covariance matrix of the states and measurements. The linearization method can be derived as:

$$\mathbf{x}_k = \mathbf{f}(\mathbf{u}_k, \mathbf{x}_{k-1}, \mathbf{w}_k) \quad (23)$$

$$\mathbf{z}_k = \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k) \quad (24)$$

$$\begin{aligned} \bar{\mathbf{x}}_k &= \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k) \\ &\approx \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \hat{\mathbf{w}}_k = \mathbf{0}) + \left. \frac{\partial \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k)}{\partial \mathbf{x}_{k-1}} \right|_{\mathbf{x}_{k-1}=\hat{\mathbf{x}}_{k-1}, \mathbf{w}_k=\mathbf{0}} (\mathbf{x}_{k-1} - \hat{\mathbf{x}}_{k-1}) \\ &\quad + \left. \frac{\partial \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k)}{\partial \mathbf{w}_k} \right|_{\mathbf{x}_{k-1}=\hat{\mathbf{x}}_{k-1}, \mathbf{w}_k=\mathbf{0}} \mathbf{w}_k \\ &= \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \hat{\mathbf{w}}_k = \mathbf{0}) + \mathbf{A}_k(\mathbf{x}_{k-1} - \hat{\mathbf{x}}_{k-1}) + \mathbf{W}_k \mathbf{w}_k \end{aligned} \quad (25)$$

$$\begin{aligned} \mathbf{z}_k &= \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k) \\ &\approx \mathbf{h}(\hat{\mathbf{x}}_k, \mathbf{0}) + \left. \frac{\partial \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)}{\partial \mathbf{x}_k} \right|_{\mathbf{x}_k=\hat{\mathbf{x}}_k, \mathbf{v}_k=\mathbf{0}} (\mathbf{x}_k - \hat{\mathbf{x}}_k) + \left. \frac{\partial \mathbf{h}(\mathbf{x}_k, \mathbf{v}_k)}{\partial \mathbf{v}_k} \right|_{\mathbf{x}_k=\hat{\mathbf{x}}_k, \mathbf{v}_k=\mathbf{0}} \mathbf{v}_k \\ &= \mathbf{h}(\hat{\mathbf{x}}_k, \mathbf{0}) + \mathbf{H}_k(\mathbf{x}_k - \hat{\mathbf{x}}_k) + \mathbf{V}_k \mathbf{v}_k \end{aligned} \quad (26)$$

where:

- $\mathbf{x}_k$: State vector at time k ($n \times 1$).
- $\mathbf{u}_k$: Control input vector at time k ($l \times 1$).
- $\mathbf{z}_k$: Measurement (observation) vector at time k ($m \times 1$).
- $\mathbf{f}(\cdot)$: Nonlinear motion (process) model, propagates the state.
- $\mathbf{h}(\cdot)$: Nonlinear observation (measurement) model, predicts measurement from state.

- $\hat{\mathbf{x}}_{k-1}$: State estimate at previous step ($n \times 1$).
- $\bar{\mathbf{x}}_k$: Predicted state (a priori estimate) at time k .
- $\mathbf{w}_k$: Process noise vector at time k (typically Gaussian, $n \times 1$).
- $\mathbf{v}_k$: Measurement noise vector at time k (typically Gaussian, $m \times 1$).
- $\mathbf{A}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{x}_{k-1}}$: Jacobian of process model w.r.t. previous state, evaluated at $(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \mathbf{0})$.
- $\mathbf{W}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{w}_k}$: Jacobian of process model w.r.t. process noise, evaluated at $(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \mathbf{0})$.
- $\mathbf{H}_k = \frac{\partial \mathbf{h}}{\partial \mathbf{x}_k}$: Jacobian of observation model w.r.t. current state, evaluated at $(\hat{\mathbf{x}}_k, \mathbf{0})$.
- $\mathbf{V}_k = \frac{\partial \mathbf{h}}{\partial \mathbf{v}_k}$: Jacobian of observation model w.r.t. measurement noise, evaluated at $(\hat{\mathbf{x}}_k, \mathbf{0})$.

Using these approximations, the Extended Kalman Filter (EKF) gives us the following algorithm:

Algorithm Extended Kalman Filter($\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \mathbf{w}_k$)

$$\hat{\hat{\mathbf{x}}}_k = \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \mathbf{0}); \quad (27)$$

$$\bar{\mathbf{P}}_k = \mathbf{A}_k \mathbf{P}_{k-1} \mathbf{A}_k^\top + \mathbf{W}_k \mathbf{Q}_k \mathbf{W}_k^\top; \quad (28)$$

$$\mathbf{K}_k = \bar{\mathbf{P}}_k \mathbf{H}_k^\top (\mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{V}_k \mathbf{R}_k \mathbf{V}_k^\top)^{-1}; \quad (29)$$

$$\hat{\mathbf{x}}_k = \hat{\hat{\mathbf{x}}}_k + \mathbf{K}_k (\mathbf{z}_k - \mathbf{h}(\hat{\hat{\mathbf{x}}}_k, \mathbf{0})); \quad (30)$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \bar{\mathbf{P}}_k; \quad (31)$$

where:

- $\hat{\mathbf{x}}_{k-1}$: A posteriori state estimate at time step $k-1$ ($n \times 1$)
- $\hat{\hat{\mathbf{x}}}_k$: A priori (predicted) state estimate at time step k ($n \times 1$)
- $\hat{\mathbf{x}}_k$: A posteriori state estimate at time step k ($n \times 1$)
- $\mathbf{P}_{k-1}$: A posteriori error covariance matrix at time step $k-1$ ($n \times n$)
- $\bar{\mathbf{P}}_k$: A priori (predicted) error covariance matrix at time step k ($n \times n$)
- $\mathbf{P}_k$: A posteriori error covariance matrix at time step k ($n \times n$)
- $\mathbf{Q}_k$: Process noise covariance matrix at time step k ($n \times n$)
- $\mathbf{R}_k$: Measurement noise covariance matrix at time step k ($m \times m$)
- $\mathbf{K}_k$: Kalman gain matrix at time step k ($n \times m$)
- $\mathbf{I}$: Identity matrix of appropriate dimension ($n \times n$)

Algorithm 2 Extended Kalman Filter (EKF) Algorithm

$\hat{\mathbf{x}}_k \leftarrow \hat{\mathbf{x}}_0, \bar{\mathbf{P}}_k \leftarrow \bar{\mathbf{P}}_0$ ▷ Initial State mean and covariance

while true **do**

Predict Next State Mean

$$\hat{\mathbf{x}}_k = \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k, \mathbf{0}); \quad (32)$$

Predict Next State Covariance

$$\bar{\mathbf{P}}_k = \mathbf{A}_k \mathbf{P}_{k-1} \mathbf{A}_k^\top + \mathbf{W}_k \mathbf{Q}_k \mathbf{W}_k^\top; \quad (33)$$

if Measurement ($\mathbf{z}_k$) available **then**

Calculate Kalman Gain ($\mathbf{K}$)

$$\mathbf{K}_k = \bar{\mathbf{P}}_k \mathbf{H}_k^\top (\mathbf{H}_k \bar{\mathbf{P}}_k \mathbf{H}_k^\top + \mathbf{V}_k \mathbf{R}_k \mathbf{V}_k^\top)^{-1} \quad (34)$$

Update State Mean

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k + \mathbf{K}_k (\mathbf{z}_k - \mathbf{h}(\hat{\mathbf{x}}_k, \mathbf{0})) \quad (35)$$

Update State Covariance

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}_k) \bar{\mathbf{P}}_k \quad (36)$$

end if

end while

On similar lines as KF, the (32) and (33) can be considered as a prediction step where we are trying to predict the next state of the system using the input and the previous state. The (34 - 36) can be considered as the update step which refine and update the prediction based on the observation.

Using either KF or EKF, based on the particular scenario, a simple pseudo code for implementing a localization loop for robots can be written as follows:

Algorithm 3 Method **LocalizationLoop**

```

 $[\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1}] \leftarrow [\mathbf{x}_0, \mathbf{P}_0]$  ▷ Initial State mean and covariance
 $k \leftarrow 1$  ▷ Initial k value
while true do
     $[\hat{\mathbf{x}}_k, \mathbf{P}_k] \leftarrow \text{Localize}(\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1})$  ▷ Localization
     $k \leftarrow k + 1$ 
end while

```

Algorithm 4 Method **Localize**($\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1}$)

```

 $[\mathbf{u}_k, \mathbf{Q}_k] \leftarrow \text{GetInput}();$  ▷ Get Input Data
 $[\hat{\mathbf{x}}_k, \bar{\mathbf{P}}_k] \leftarrow \text{Prediction}(\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1}, \mathbf{u}_k, \mathbf{Q}_k);$ 
if Measurement ( $\mathbf{z}_k$ ) available then
     $[\mathbf{z}_k, \mathbf{R}_k, \mathbf{H}_k, \mathbf{V}_k] \leftarrow \text{GetMeasurements}();$ 
     $[\hat{\mathbf{x}}_k, \mathbf{P}_k] \leftarrow \text{Update}(\hat{\mathbf{x}}_k, \bar{\mathbf{P}}_k, \mathbf{z}_k, \mathbf{R}_k, \mathbf{H}_k, \mathbf{V}_k);$ 
end if
return ( $\hat{\mathbf{x}}_k, \mathbf{P}_k$ )

```

where:

- **LocalizationLoop()**: Main iterative process for recursive Bayesian localization.
- **Localize**($\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1}$): Executes one full localization (prediction,update) iteration.
- *GetInput()*: Returns current control and uncertainty.
- *Prediction*($\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1}, \mathbf{u}_k, \mathbf{Q}_k$): Using previous state, covariance, input, and noise to predict the next state and then return predicted state/covariance.
- *GetMeasurements()*: Return current measurement and related matrices.
- *Update*($\hat{\mathbf{x}}_k, \bar{\mathbf{P}}_k, \mathbf{z}_k, \mathbf{R}_k, \mathbf{H}_k, \mathbf{V}_k$): Returns posterior state/covariance using predicted values and measurements.

4 EKF for 3-DOF Differential Drive Robot with Input Displacement

According to the previous section, the required equation for the EKF algorithm must be formulated. First, the state equation of the system needs to be defined. In this configuration,

only the position and orientation of the robot are the state of interest since there are no assumptions about the velocity in this model.

$$\mathbf{x}_k = \boldsymbol{\eta}_k = \begin{bmatrix} x_k \\ y_k \\ \psi_k \end{bmatrix} \quad (37)$$

4.1 Motion model of 3-DOF Differential Drive Robot with Input Displacement

Next, The motion model for the differential drive mobile robot with the encoder input can be written as

$$\mathbf{x}_k = \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k) \quad (38)$$

$$= \boldsymbol{\eta}_{k-1} \oplus (\mathbf{B}(\mathbf{u}_k + \mathbf{w}_k) \cdot \Delta t) \quad (39)$$

Where, $\mathbf{u}_k = [n_L, n_R]^\top$ is the encoder output, $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}_k) = [w_{n_L,k}, w_{n_R,k}]^\top$ is the encoder's noise that have zero mean and $\mathbf{Q}_k = \text{diag}(\sigma_{n_L,k}^2, \sigma_{n_R,k}^2)$, and $\mathbf{B}$ is the mapping matrix that map the encoder output to velocity of the robot express in robot frame $\{B\}$ which can be written as.

$$\mathbf{B} = \begin{bmatrix} \frac{M}{2} & \frac{M}{2} \\ 0 & 0 \\ -\frac{M}{L} & \frac{M}{L} \end{bmatrix} \quad (40)$$

Where, $M = \frac{2\pi \cdot f_{encoder} \cdot R_w}{n_{2\pi}}$ is the constant value that is used to map the number of pulses obtained from the encoder to the velocity of the wheel. Then, after substitution the (40) to (39), the complete equation can be obtained.

$$\begin{aligned} \mathbf{x}_k &= \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k) \\ &= \begin{bmatrix} x_{k-1} + \underbrace{\left(\frac{M}{2}\right)((n_L + w_{n_L,k}) + (n_R + w_{n_R,k})) \cos(\psi_{k-1}) \cdot \Delta t}_{u_{noise,k}} \\ y_{k-1} + \underbrace{\left(\frac{M}{2}\right)((n_L + w_{n_L,k}) + (n_R + w_{n_R,k})) \sin(\psi_{k-1}) \cdot \Delta t}_{u_{noise,k}} \\ \psi_{k-1} + \underbrace{\left(\frac{M}{L}\right)(-(n_L + w_{n_L,k}) + (n_R + w_{n_R,k})) \cdot \Delta t}_{r_{noise,k}} \end{bmatrix} \quad (41) \\ &= \begin{bmatrix} x_{k-1} + u_{noise,k} \cdot \cos(\psi_{k-1}) \cdot \Delta t \\ y_{k-1} + u_{noise,k} \cdot \sin(\psi_{k-1}) \cdot \Delta t \\ \psi_{k-1} + r_{noise,k} \cdot \Delta t \end{bmatrix} \quad (42) \end{aligned}$$

For the kinematic motion model using pose composition (42), the Jacobians concerned with motion model are calculated as follows:

State Jacobian $\mathbf{A}_k$

The matrix $\mathbf{A}_k$ captures how a small change in the previous state $\mathbf{x}_{k-1}$ affects the predicted state. For the pose compounding operation, the partial derivatives yield:

$$\mathbf{A}_k = \frac{\partial \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k)}{\partial \mathbf{x}_{k-1}} \quad (43)$$

$$= \begin{bmatrix} 1 & 0 & -u_{noise,k} \cdot \sin(\psi_{k-1}) \cdot \Delta t \\ 0 & 1 & u_{noise,k} \cdot \cos(\psi_{k-1}) \cdot \Delta t \\ 0 & 0 & 1 \end{bmatrix} \quad (44)$$

Noise Jacobian $\mathbf{W}_k$

The matrix $\mathbf{W}_k$ represents how process noise propagates through the nonlinear motion model. For a displacement applied in the robot frame, the effect in the global frame is obtained by rotating the noise vector by the robot's orientation:

$$\mathbf{W}_k = \frac{\partial \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_k, \mathbf{w}_k)}{\partial \mathbf{w}_k} \quad (45)$$

$$= \begin{bmatrix} \frac{M}{2} \Delta T \cos(\psi_{k-1}) & \frac{M}{2} \Delta T \cos(\psi_{k-1}) \\ \frac{M}{2} \Delta T \sin(\psi_{k-1}) & \frac{M}{2} \Delta T \sin(\psi_{k-1}) \\ -\frac{M}{L} \Delta T & \frac{M}{L} \Delta T \end{bmatrix} \quad (46)$$

This Jacobian accounts for the rotation of the noise vector from the robot frame into the world frame as part of the compounding.

4.2 Measurement Model of 3-DOF Differential Drive Robot with Input Displacement

The measurement (observation) model for the 3-DOF differential drive robot with input displacement is implemented using a compass sensor, which measures the current yaw angle of the robot. The observation is modeled as:

$$\mathbf{z}_{com,k} = \psi_k + \mathbf{v}_{com,k} \quad (47)$$

$$\mathbf{z}_{com,k} = \mathbf{h}(\mathbf{x}_k) + \mathbf{v}_{com,k} \quad (48)$$

$$\mathbf{h}(\mathbf{x}_k) = [0 \quad 0 \quad 1] \quad (49)$$

Where, $\mathbf{z}_{com,k}$ is the output of the compass that consists of the robot's heading with noise. The noise in the compass reading is modeled using a random variable variable $\mathbf{v}_{com,k} \sim \mathcal{N}(0, \mathbf{R}_{com,k})$ and $\mathbf{R}_{com,s_k} = \sigma_{com}^2$.

In this simulation system, If a new compass value is available, the yaw value is provided as a 1×1 measurement vector; otherwise, the update step of the localization is not called in the localization loop (observation is not available) and only prediction happens. The corresponding measurement covariance R_k is also given as a 1×1 matrix.

The measurement Jacobian is:

$$\mathbf{H}_k = [0 \quad 0 \quad 1] \quad (50)$$

This matrix is used to map the state to the measurement space. In this case, the H matrix is very simple because only the yaw (third state component) is observed directly.

Additionally, the Jacobian with respect to measurement noise is:

$$\mathbf{V}_k = [1] \quad (51)$$

This model is typical for compass (yaw) sensors in mobile robotics, and its linearity with respect to the state allows the use of the standard EKF update equations without further approximation.

4.3 Estimate Next Robot State in Localization Loop

After the define all required variables and matrices, the Localizationloop algorithm 3 can be perform to localize the robot. At each iteration of the localization loop, the EKF estimates the next robot state by propagating the previous state forward in time using the motion (prediction) model, and - when available - refining this prediction using new sensor measurements.

For the prediction step, The estimator utilizes velocity information derived from wheel encoder readings ($\mathbf{u}_k$). These encoder counts are converted to wheel velocities, then to the robot's linear and angular velocity ($u_{noise,k}, r_{noise,k}$). This process produces the robot's incremental displacement vector $\mathbf{u}_k$ and its associated covariance $\mathbf{Q}_k$, as described in Section 4.1. The core of the prediction step is propagate the robot's states ,which are random variables, though the motion model. The mean value of the states ($\hat{\mathbf{x}}$) can be done by the pose composition operation (compounding), defined by (42) which is non-linear function. On the other hand, The covariance matrix ($\mathbf{P}$) is transformed by the linear approximation matrix (44). This logic is implemented by the *Prediction*($\hat{\mathbf{x}}_{k-1}, \mathbf{P}_{k-1}, \mathbf{u}_k, \mathbf{Q}_k$) method in algorithm 4.

Then, the update step refines the predicted state $\hat{\mathbf{x}}_k$ using the newly acquired compass measurement $\mathbf{z}_k$, as defined in (47). The Kalman gain $\mathbf{K}_k$ is first computed using the (34). The updated (a posteriori) state and covariance are then given by (35) and (36), respectively. These equations are implemented int the *Update*($\hat{\mathbf{x}}_k, \bar{\mathbf{P}}_k, \mathbf{z}_k, \mathbf{R}_k, \mathbf{H}_k, \mathbf{V}_k$) method in algorithm 4.

Together, the prediction and update steps correspond to one iteration of the EKF-based localization. The prediction estimates how the robot moves given the applied control inputs and motion uncertainty, while the update adjusts this estimate according to the measured yaw from the compass sensor, thereby minimizing uncertainty. This two-stage recursive process continuously refines the robot's position and orientation estimates throughout the localization loop.

However, in general differences in the sensor sampling rate and localization update frequency can impact system performance. If the sensor reading rate exceeds the localization update rate, the discrepancy can be managed by selectively skipping update called in the filter. On the other hand, if the sensor reading rate is lower than the localization update rate, the localization algorithm must rely on previously acquired sensor data, which may increase the accumulation of estimation errors, until the new measurement data is available.

4.4 System diagram of EKF localization for 3-DOF Differential Drive Robot with Input Displacement

In summary, EKF localization with the simulation system can be illustrated in Figure ??

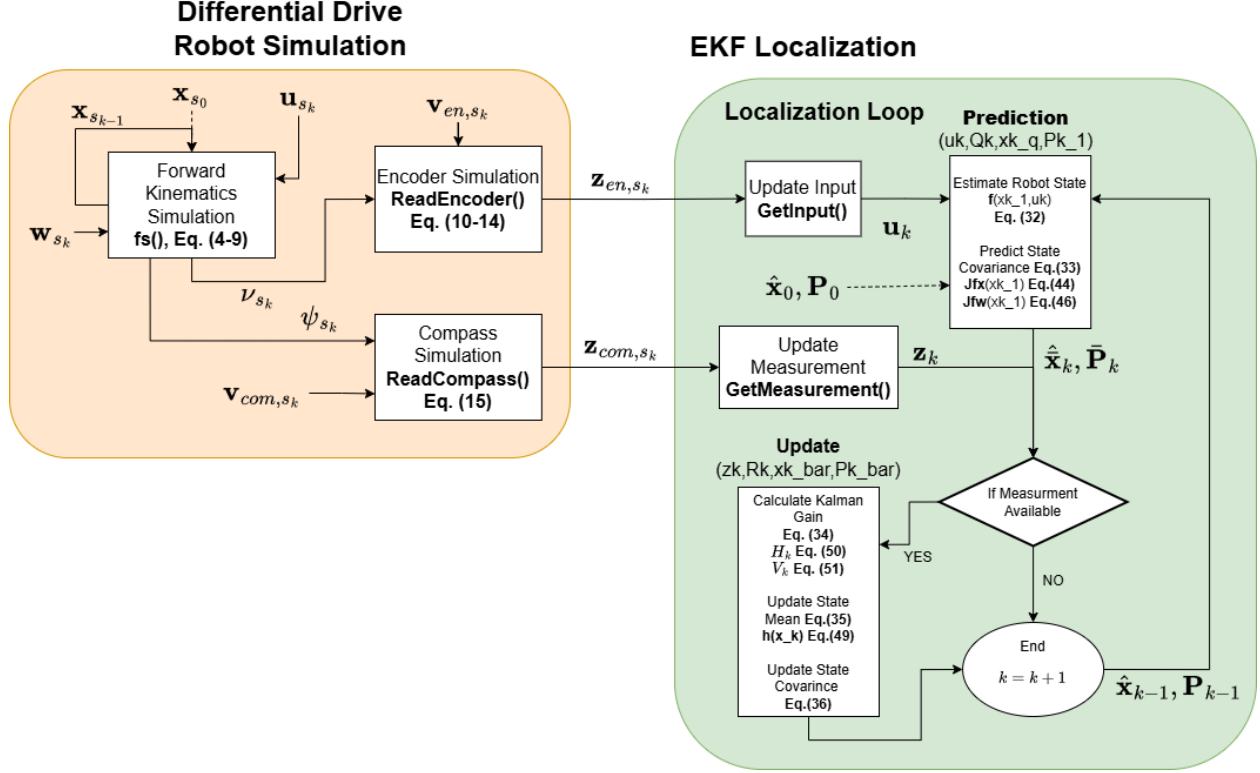


Figure 3: System diagram of EKF localization

Finally, the class implementation developed for this lab is presented in Figure 4. This diagram provides a clear overview, allowing the reader to navigate the code implementation with ease.

Class Implementation

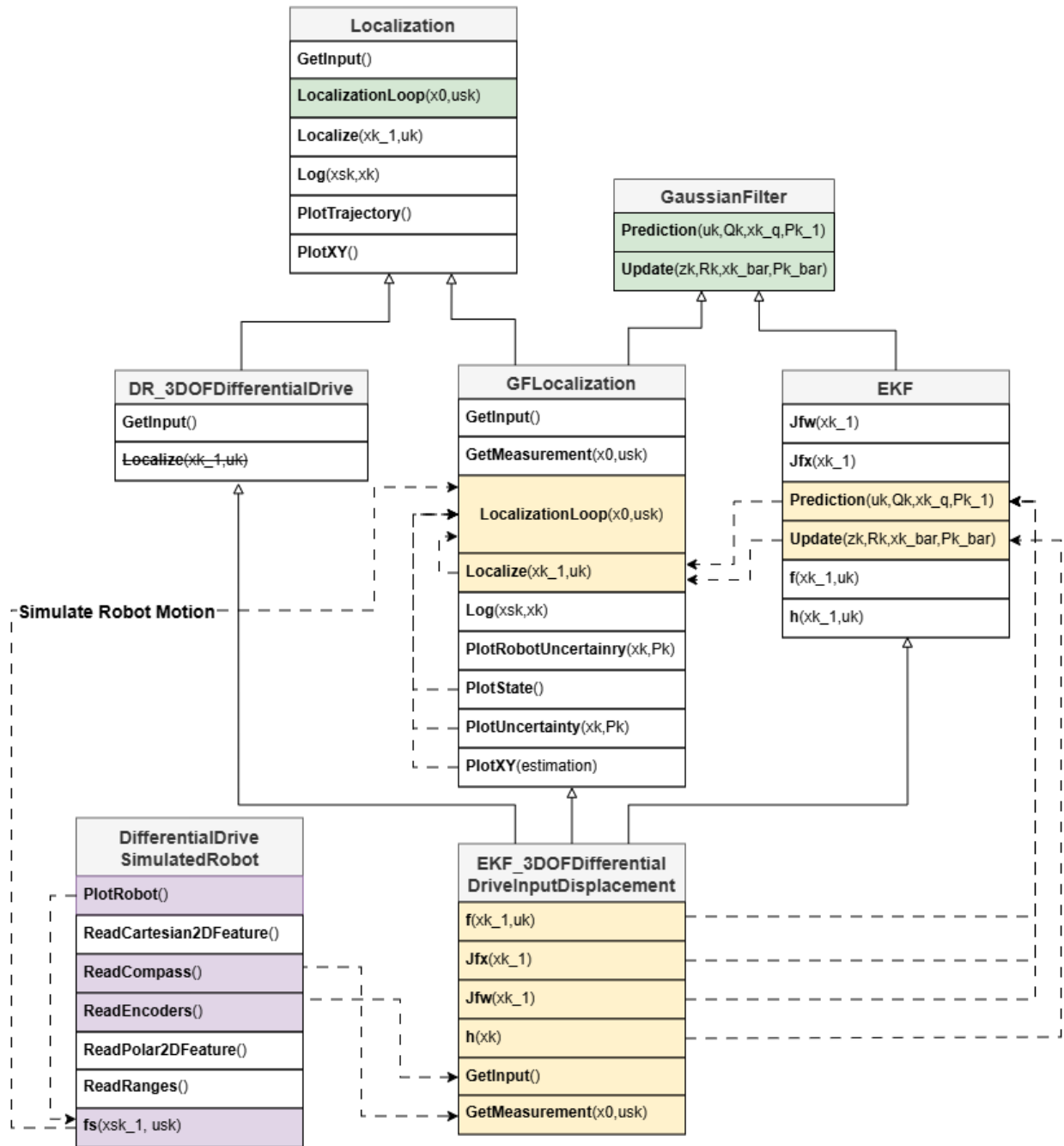


Figure 4: EKF Class Implementation

4.5 EKF Localization result

By continuously executing the localization loop to iteratively update the robot's pose, the resulting trajectory can be visualized as illustrated in Figure 5, Figure 7, Figure 9 and Figure 11 for different values of the sensor (compass in our case) frequency. The time step of single simulation step is of 0.1 second, so, if the frequency = 10, then the sensor reading is coming at every time step. If the frequency = 1, then the sensor reading is coming at every 10 time steps of the simulation, which is equivalent to 1 second. For sensor reading to come at every 10 seconds, which means at every 100 time steps of the simulation, we will need to set the frequency to 0.1. For sensor reading to come after all the time steps of the simulation, that is, after 5000 time steps, which is equivalent to $5000 * 0.1 = 500$ seconds, the frequency should be set to $1/500$, the situation in which there will be no update step ever and the filter will purely work on the basis of prediction.

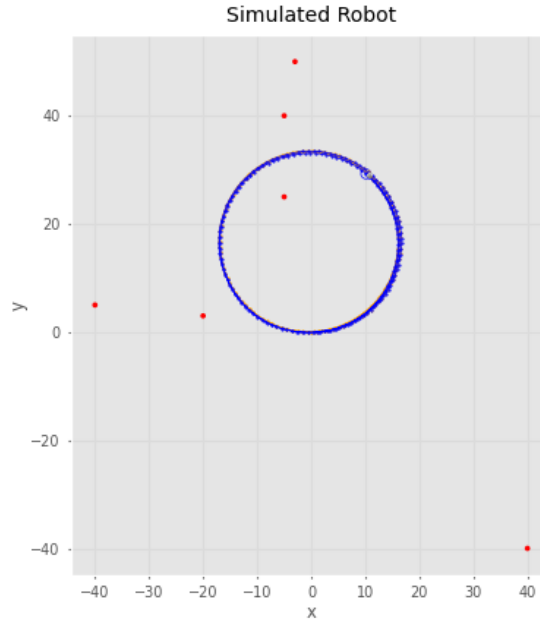


Figure 5: Trajectory Comparison: Simulated Robot vs. EKF Localization for sensor frequency=10

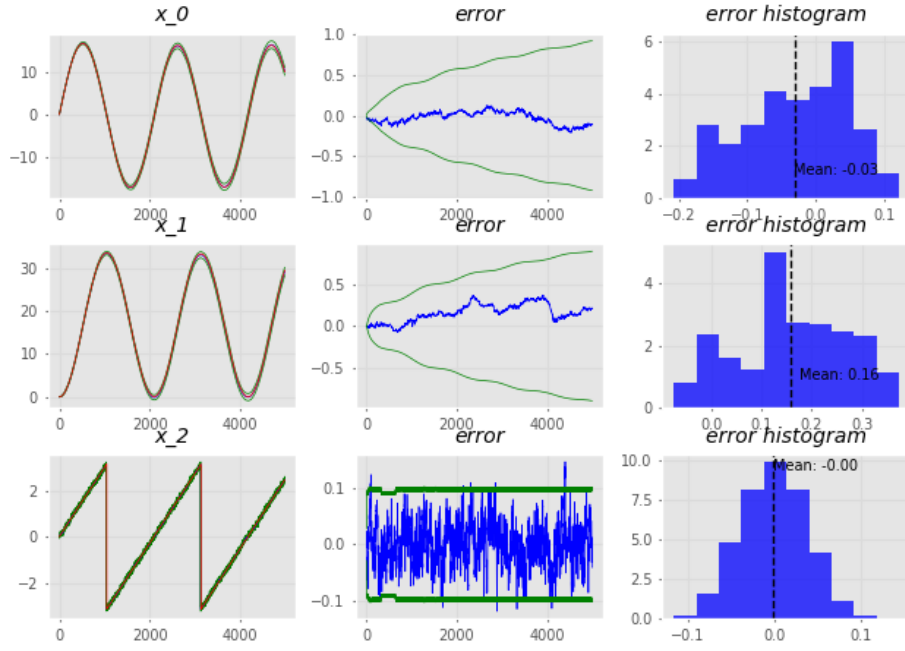


Figure 6: Results for EKF Localization for sensor frequency=10

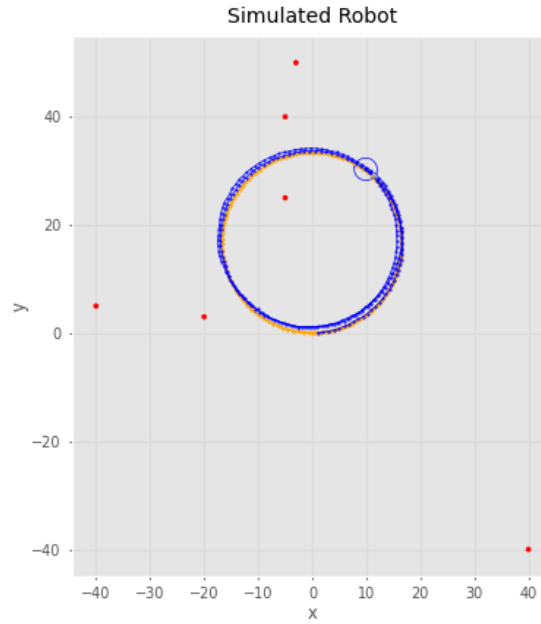


Figure 7: Trajectory Comparison: Simulated Robot vs. EKF Localization for sensor frequency=1

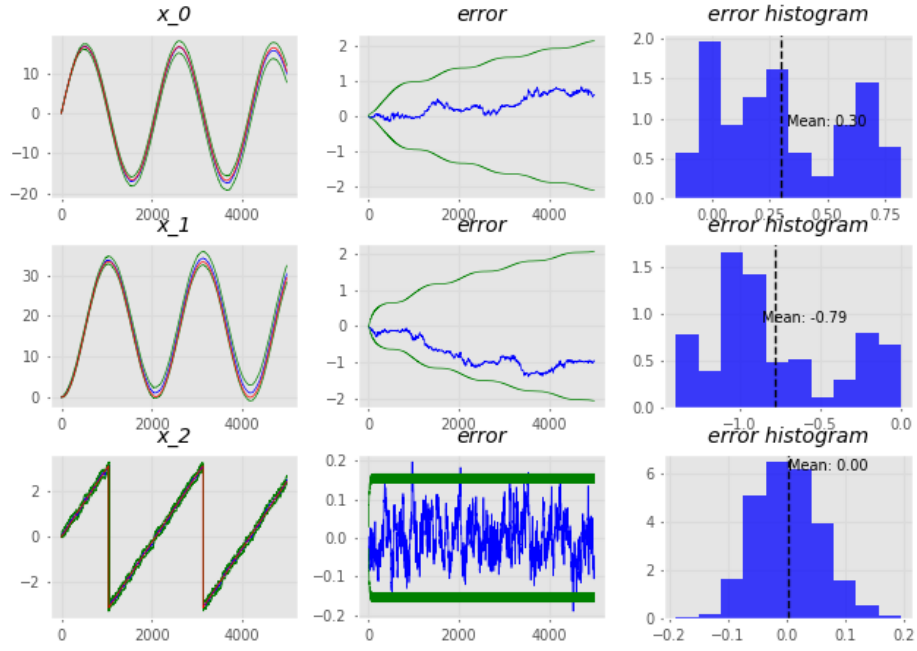


Figure 8: Results for EKF Localization for sensor frequency=1

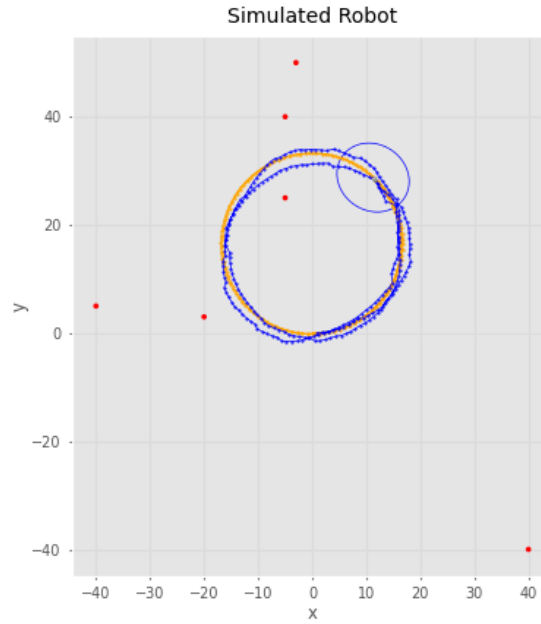


Figure 9: Trajectory Comparison: Simulated Robot vs. EKF Localization for sensor frequency=0.1

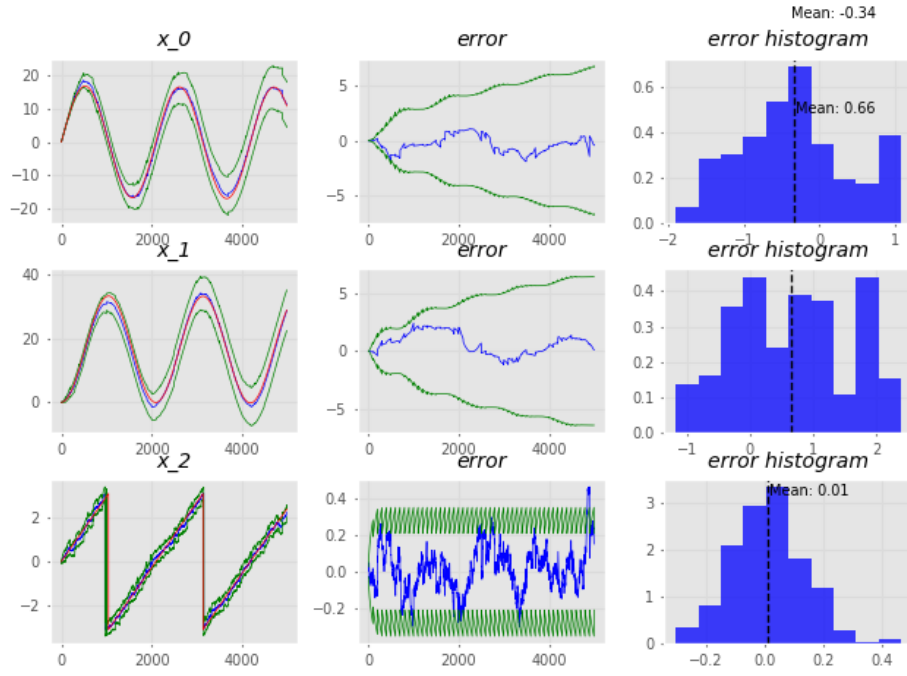


Figure 10: Results for EKF Localization for sensor frequency=0.1

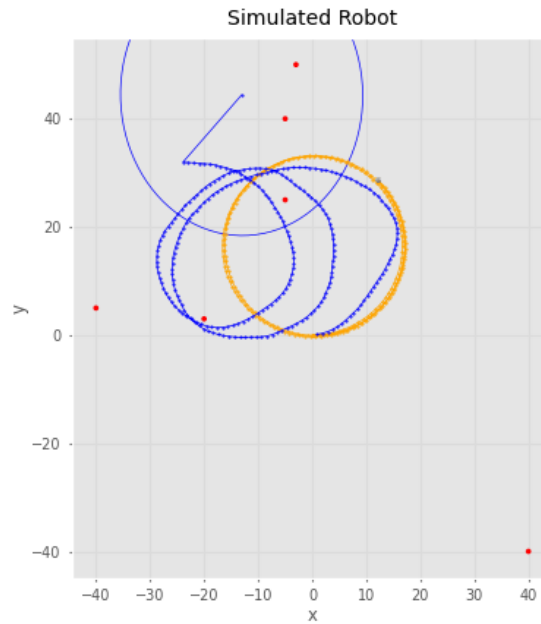


Figure 11: Trajectory Comparison: Simulated Robot vs. EKF Localization for sensor frequency=1/500

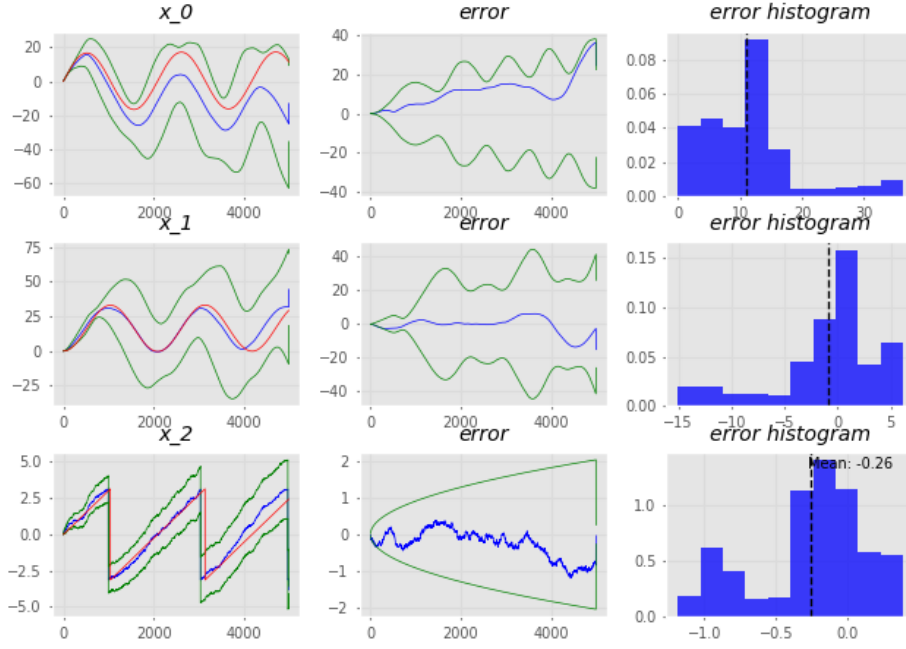


Figure 12: Results for EKF Localization for sensor frequency=1/500

5 Issues

A critical challenge encountered during the implementation of the Extended Kalman Filter (EKF) for 3-DOF localization involves the handling of angular quantities, specifically the robot’s yaw (orientation), ψ . This issue, known as “angle wrapping”, can lead to large, unbounded state update errors if not properly addressed.

The problem originates in the EKF’s **update step**, which relies on the *innovation* (also called the measurement residual), y_k . The innovation represents the difference between the actual measurement z_k and the filter’s predicted measurement $h(x_k)$:

$$y_k = z_k - h(x_k) \quad (52)$$

In our system, z_k is the compass reading for the yaw, and $h(x_k)$ is the expected measurement data that is the filter’s current estimate of the yaw. Angular measurements are cyclical, meaning that an heading of the robot is wrapped around $(\pi, -\pi)$. A standard subtraction, however, fails to capture this property.

Consider a scenario where the robot is pointing almost due West:

- The true yaw (and measurement z_k) is 3.14 rad (almost $+\pi$).
- The filter’s predicted yaw $h(x_k)$ is -3.14 rad (almost $-\pi$).

Physically, the filter’s estimate is extremely accurate, with an error close to zero. However, the innovation calculation (52) yields:

$$y_k = 3.14 - (-3.14) = 6.28 \approx 2\pi \quad (53)$$

The filter incorrectly computes a massive error of 2π radians (a full circle). This erroneous innovation is then used to "correct" the state estimate, as shown in the EKF update equation:

$$\hat{x}_k = \hat{\hat{x}}_k + K_k y_k \quad (54)$$

Where K_k is the Kalman gain. The large, incorrect innovation y_k is multiplied by the gain and results in a massive, invalid correction to the state vector x_k . This destabilizes the filter and causes the state estimate to diverge rapidly from the true pose, as was observed in our initial, divergent plots.

The Solution

The solution is to make sure that the angular component of the innovation are "wrap" to the interval $[-\pi, \pi]$ *before* it is used in the update equation (54). This is achieved by applying a wrapping function, $\text{wrap}(\cdot)$, to the innovation:

$$\text{wrap}(\theta) = ((\theta + \pi) \bmod 2\pi) - \pi \quad (55)$$

The corrected update process first calculates the raw innovation as in (52), but then immediately wraps it:

$$y_{k,\text{wrapped}} = \text{wrap}(y_k) \quad (56)$$

Using our previous example, the wrapped innovation becomes:

$$y_{k,\text{wrapped}} = \text{wrap}(6.28) \approx 0.0 \quad (57)$$

This near-zero value correctly reflects the (near-zero) error between the prediction and the measurement. This wrapped innovation is then used in the final update:

$$x_k = x_{k,\text{predicted}} + K_k \cdot y_{k,\text{wrapped}} \quad (58)$$

By implementing this correction, the filter properly interprets angular differences, ensuring that only the "shortest path" or "small side angle" angular error is used for the correction. This modification stabilized the filter and allowed it to correctly track the robot's true pose.

6 Conclusions

The laboratory exercise successfully demonstrated the design, implementation, and evaluation of an Extended Kalman Filter (EKF) localization system for a differential drive robot. Through theoretical formulation, simulation, and experimentation, the work validated EKF's ability to maintain accurate trajectory tracking under nonlinear kinematic conditions when provided with consistent sensor updates. The implementation process revealed the importance of correctly formulating motion and measurement models, handling

uncertainties through Jacobian-based linearization, and managing angular computations to ensure filter stability.

Analysis across multiple sensor frequencies highlighted the essential trade-off between update rate and localization accuracy. High-frequency updates maintained bounded error and robust performance, while reduced sensor availability led to cumulative drift and degraded trajectory estimation. The study also emphasized critical insights into recursive Bayesian estimation, numerical stability in angle normalization, and the dependency of EKF performance on measurement diversity. Future extensions incorporating landmark-based position updates and probabilistic data association are expected to enhance accuracy and robustness in more complex environments.

References

- [1] Pere Ridao. "Probabilistic Robot Localization & Mapping". Unpublished manuscript. 2025